

Comunicación Wi-Fi con Esp-01

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

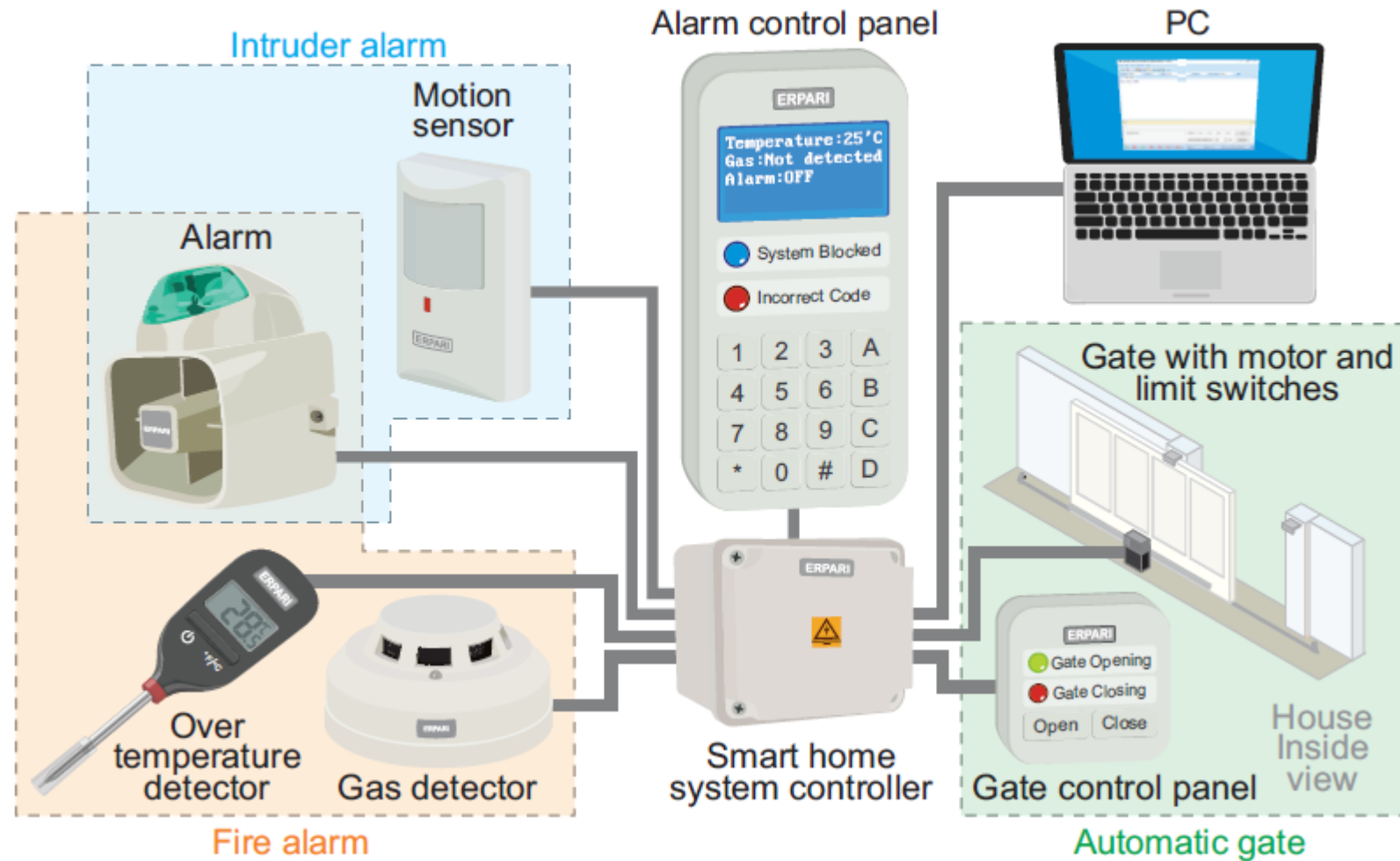
2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Periféricos y Comunicaciones en Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

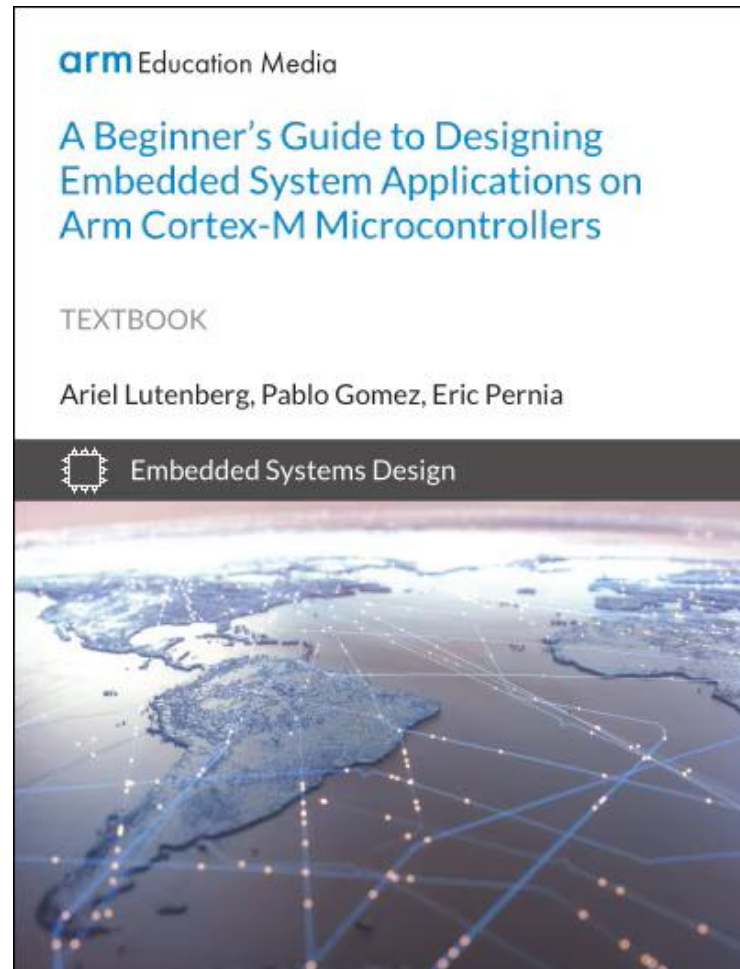
Temas para la semana 3



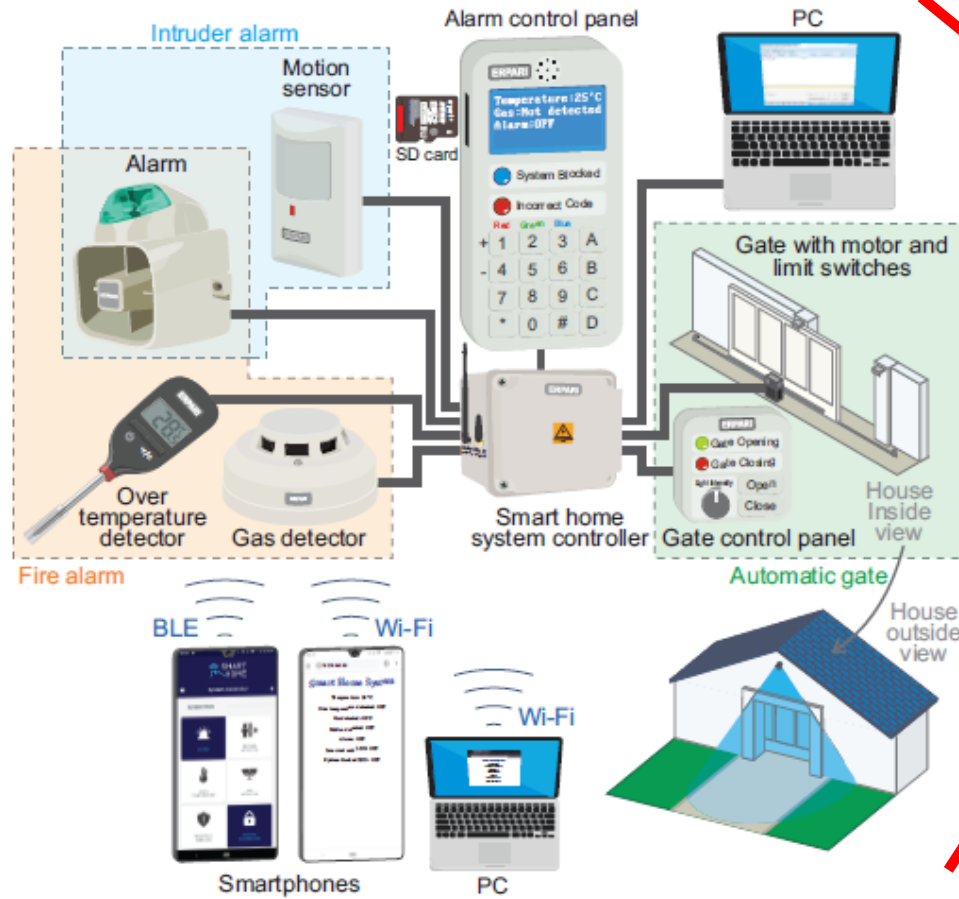
Temas para la semana 6.

6	Contenido de las clases para la semana 6. Teoría: Integración de módulos <u>Wi-Fi</u> a la placa NUCLEO-F446RE. Servidor web embebido basado en una conexión <u>Wi-Fi</u>. Práctica: Despliegue de un servidor web para monitoreo remoto de variables en tiempo real. <u>Node-RED</u>: empleo de esta herramienta para la implementación de un <u>Dashboard</u> profesional del sistema.	Será informada por el docente a cargo.	1, 2, 3 y 4.
7	Contenido de la clase 7. Teoría: Node-RED como puente entre la placa NUCLEO-F446RE y las <u>APIs</u> de consumo. Integrar el envío de mensajes por e-mail. Envío de datos haciendo uso de aplicaciones móviles de comunicación. Práctica: Desarrollo de un sistema para monitoreo y alarmas. Se utilizan los datos de los sensores conectados a la placa NUCLEO-F446RE, para que se envíen por e-mail y diferentes aplicaciones de mensajería móvil.	Será informada por el docente a cargo.	1, 2, 3 y 4.
8	Exposición de los trabajos finales.		

Libro de Referencia



Libro de referencia.



arm Education Media

A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers

TEXTBOOK

Ariel Lutenberg, Pablo Gomez, Eric Pernia

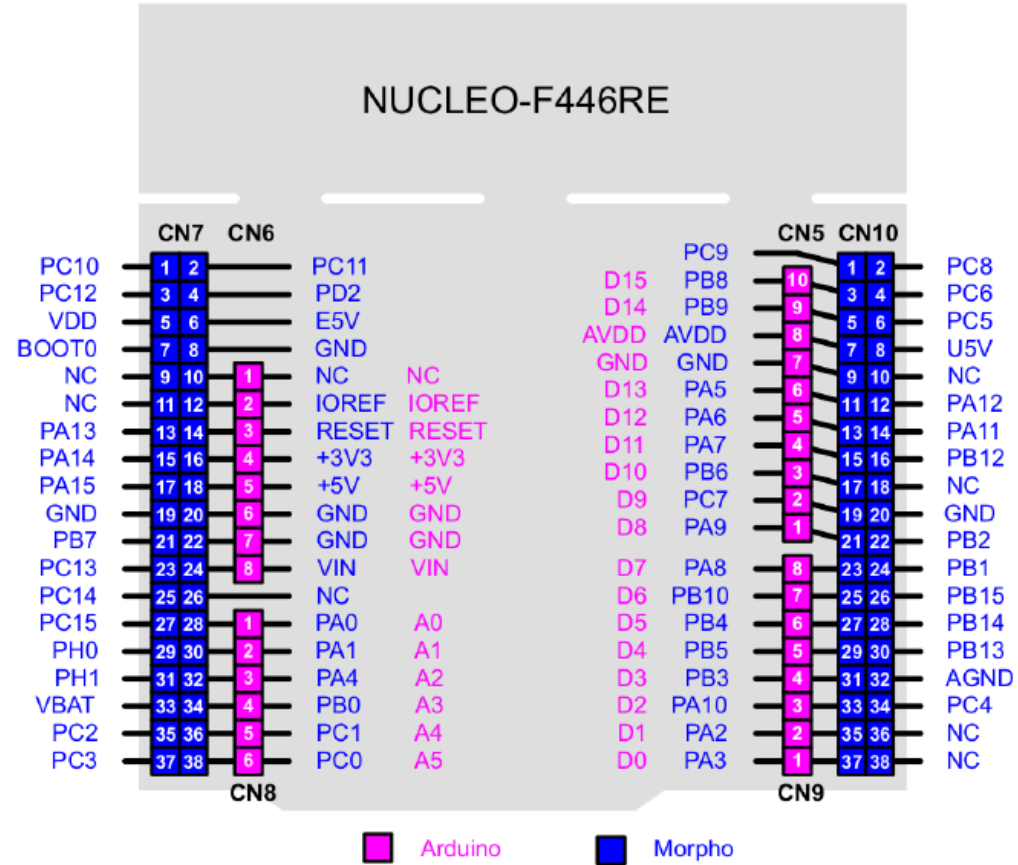


Embedded Systems Design

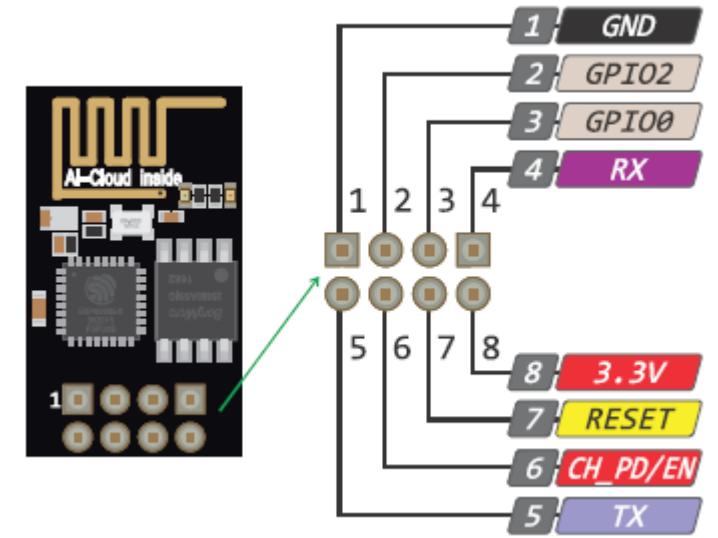


<https://www.arm.com/resources/education/books/designing-embedded-systems>

Repaso: Wi-Fi.

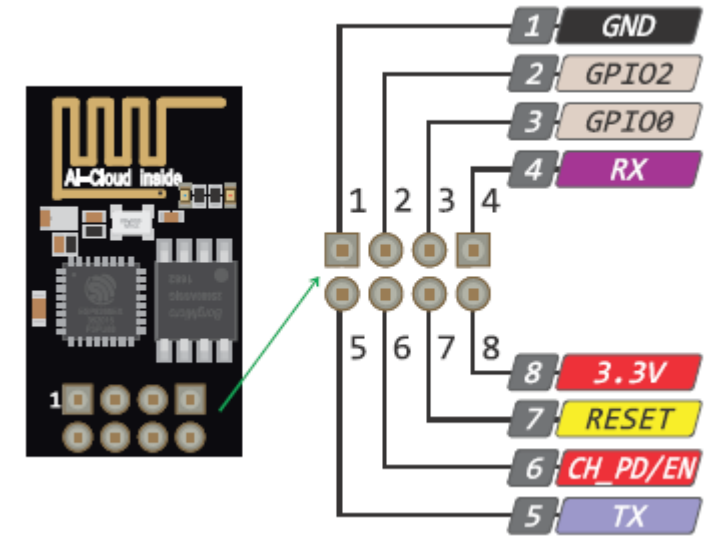


Repaso: Wi-Fi.



- Describir la conexión de un módulo Wi-Fi a la placa NUCLEO.
- Desarrollar programas para implementar páginas web desde el microcontrolador.
- Comprender los fundamentos de los comandos AT y las conexiones TCP/IP.

Repaso: Wi-Fi.



- Algunas ventajas de la comunicación inalámbrica:
- Superar la limitación de alcance del Bluetooth Low Energy (pocos metros).
- Permitir el acceso desde cualquier dispositivo con navegador web (PC o Smartphone).
- Lograr mayores tasas de transferencia y entrega de datos verificada.

Repaso: Wi-Fi.



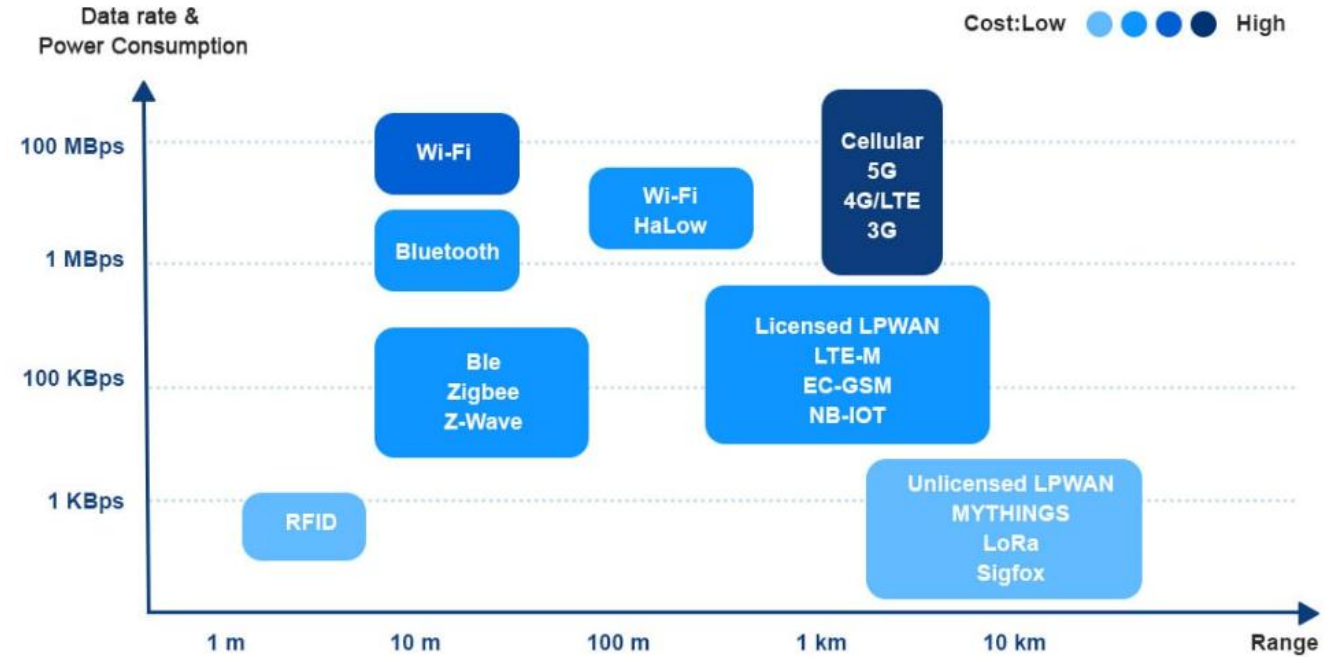
- Bluetooth Low Energy (BLE): la tasa de bits en la capa física varia entre 1 Mbit/s y 2 Mbit/s, dependiendo de la versión de BLE utilizada.
- En condiciones reales el rendimiento es menor; con el módulo HM-10 opera a solo 9600 bps debido a la interfaz UART empleada.

Repaso: Wi-Fi.



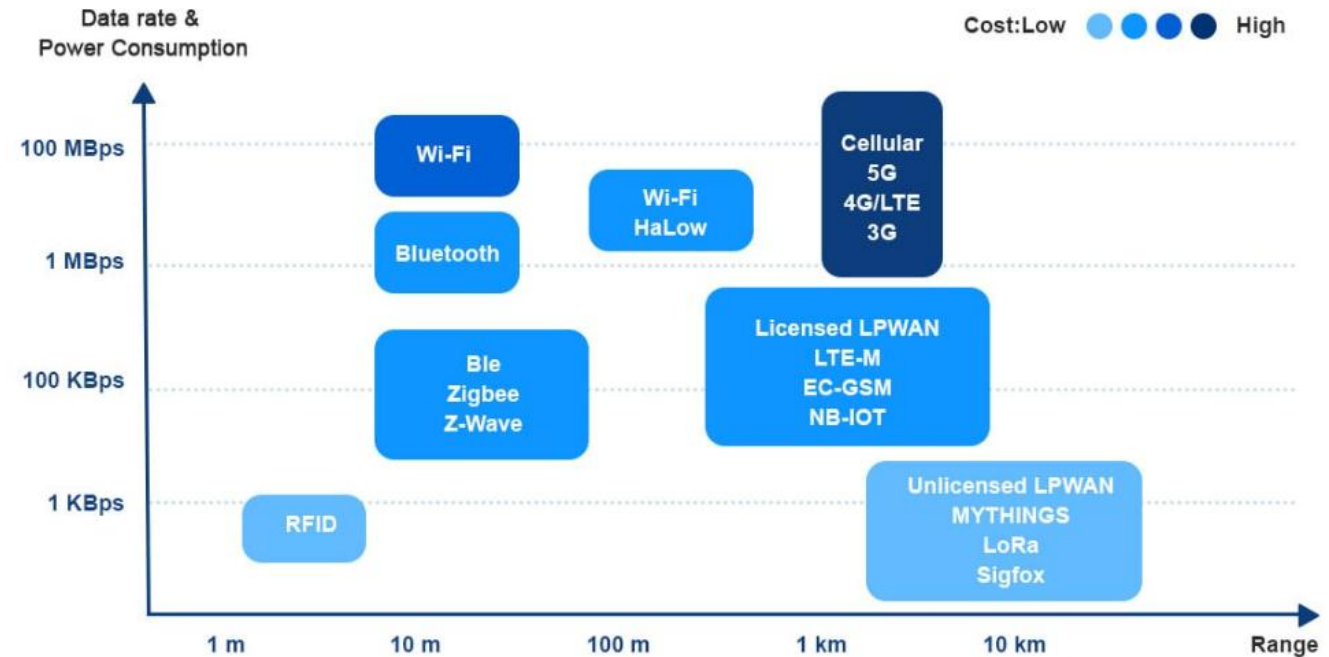
- Wi-Fi (IEEE 802.11): las velocidades varían drásticamente según la generación;
- Wi-Fi 1 (802.11b): De 1 a 11 Mbit/s.
- Wi-Fi 3 (802.11g): De 3 a 54 Mbit/s.
- Wi-Fi 4 (802.11n): De 72 a 600 Mbit/s (módulo ESP-01).
- Wi-Fi 6/6E: Puede alcanzar hasta 9608 Mbit/s.

Repaso: Wi-Fi.



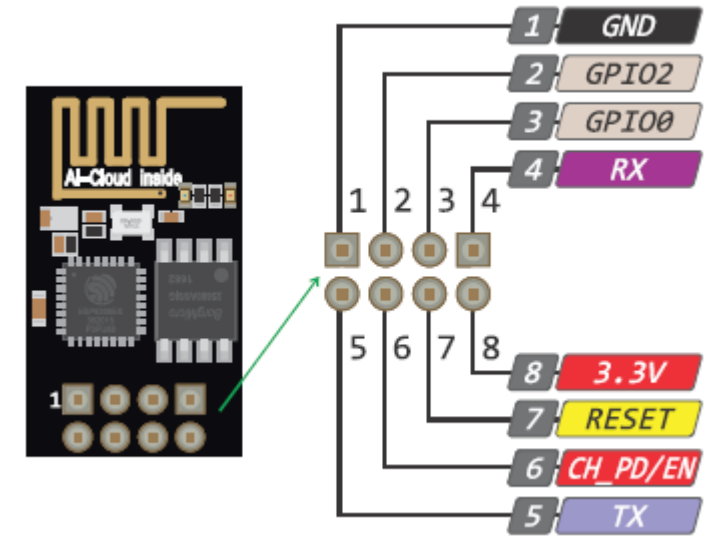
- Bluetooth: clase común (Clase 2): Hasta 10 metros. Ideal para accesorios personales como auriculares o relojes.
- Clase industrial (Clase 1): hasta 100 metros (poco común en dispositivos cotidianos).

Repaso: Wi-Fi.



- Wi-Fi: Banda de 2.4 GHz: Entre 30 y 45 metros en interiores; hasta 90 metros en exteriores.
- Banda de 5 GHz: Menor alcance (unos 15 metros en interiores) debido a que atraviesa peor los obstáculos.

Repaso: Wi-Fi.



- Las conexiones entre la placa NUCLEO y el módulo ESP-01 se resumen en la siguiente tabla.

NUCLEO board	ESP-01 module
PE_8 (UART7_TX)	RXD
PE_7 (UART7_RX)	TXD

Table 11.2 Summary of other connections that should be made to the ESP-01.

ESP-01 module	Breadboard
GND	GND
EN	3.3 V
VCC	3.3 V

Wi-Fi.

- **Estructura de funcionamiento:** La función `sprintf` toma el texto formateado (con etiquetas HTML y estilos CSS) y lo guarda dentro de una variable de tipo cadena (char array).

```
sprintf(ATcommandB, "<!DOCTYPE html><html>\n<head>\n\ <title>STM32 - ESP8266</title>\n<link\nhref=\"data:image/x-icon;base64,\ A\" rel=\"icon\" type=\"image/x-icon\"><style>\nhtml {\n  display: inline-block; margin: 0px auto; text-align: center;}\n\ body{margin-top:\n50px;}\n.button {display: block;\n\ width: 70px;\nbackground-color: #008000;\nborder:\nnone;\ncolor: white;\n\ padding: 14px 28px;\ntext-decoration: none;\nfont-size: 24px;\n\ margin: 0px auto 36px;\nborder-radius: 5px;}\n\ .button-on {background-color:\n#008000;}\n.button-on:active\ {background-color: #008000;}\n.button-off {background-color:\n#808080;}\n\ .button-off:active {background-color: #808080;}\n\ p {font-size: 14px;color:\n#808080;margin-bottom: 20px;}\n\ </style>\n</head>\n<body>\n<h1>STM32 - ESP8266</h1>");\nsprintf(ATcommandN, "<p>Light is currently on\ </p><a class=\"button button-off\"\nhref=\"/lightoff\">OFF</a>");\nsprintf(ATcommandF, "<p>Light is currently off\ </p><a class=\"button button-on\"\nhref=\"/lighton\">ON</a>");\nsprintf(ATcommandT, "</body></html>");
```

Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:
- **AT+RST\r\n**: Reinicia el módulo ESP-01 por software, sin configuraciones residuales.
- **AT+CWMODE_CUR=2\r\n**: Configura el modo de operación, haciendo que el ESP-01 funcione como su propia red Wi-Fi.



Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:

```
sprintf(ATcommand,"AT+RST\r\n");  
memset(rxBuffer,0,sizeof(rxBuffer));  
HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);  
HAL_UART_Receive (&huart4, rxBuffer, 512, 100); HAL_Delay(500);  
  
ATisOK = 0; while(!ATisOK){  
    sprintf(ATcommand,"AT+CWMODE_CUR=2\r\n");  
    memset(rxBuffer,0,sizeof(rxBuffer));  
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);  
    HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);  
    if(strstr((char *)rxBuffer,"OK")){  
        ATisOK = 1;  
    } HAL_Delay(500);  
}
```



Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:
- `AT+CWSAP_CUR="STM32","12345678",1,3,4,0\r\n`:
- "STM32": Nombre de la red Wi-Fi (SSID).
- "12345678": Contraseña.
- 1: Canal de radio Wi-Fi.
- 3: Tipo de seguridad (WPA2_PSK).
- 4 (MaxConexiones): Indica que como máximo 4 dispositivos.
- 0: El SSID es visible (no está oculto).



Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:

```
ATisOK = 0;

while(!ATisOK){
    sprintf(ATcommand,"AT+CWSAP_CUR=\"STM32\", \"12345678\",1,3,4,0\r\n");
    memset(rxBuffer,0,sizeof(rxBuffer));
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);
    HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);

    if(strstr((char *)rxBuffer,"OK")){
        ATisOK = 1;
    }
    HAL_Delay(500);
}
```

Wi-Fi.



- Comandos AT enviados al módulo ESP-01 por UART:
- **AT+CIPAP_CUR="192.168.51.1"\r\n**: Establece la dirección IP estática del servidor web en la red local del ESP-01 (192.168.51.1).
- **AT+CIPMUX=1\r\n**: Habilita las conexiones múltiples (permite que hasta 8 clientes o canales TCP se conecten al servidor de forma simultánea).

Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:

```
ATisOK = 0;
while(!ATisOK){
    sprintf(ATcommand,"AT+CIPAP_CUR=\"192.168.51.1\"\\r\\n");
    memset(rxBuffer,0,sizeof(rxBuffer));
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);
    HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);
    if(strstr((char *)rxBuffer,"OK")){
        ATisOK = 1; }
    HAL_Delay(500);
}
ATisOK = 0;
while(!ATisOK){
    sprintf(ATcommand,"AT+CIPMUX=1\\r\\n");
    memset(rxBuffer,0,sizeof(rxBuffer));
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);
    HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);
    if(strstr((char *)rxBuffer,"OK")){
        ATisOK = 1;
    } HAL_Delay(500);}
```

Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:
- **AT+CIPSERVER=1,80\r\n**: Inicia el servidor web escuchando en el puerto estándar 80 (el puerto HTTP por defecto).

```
ATisOK = 0;
while(!ATisOK){
    sprintf(ATcommand,"AT+CIPSERVER=1,80\r\n");
    memset(rxBuffer,0,sizeof(rxBuffer));
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);
    HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);
    if(strstr((char *)rxBuffer,"OK")){
        ATisOK = 1;
    } HAL_Delay(500);
}
```

Wi-Fi.

- Identificación del Canal: cuando un dispositivo (PC) abre la página web, el ESP-01 le avisa al STM32 enviando un texto que empieza con +IPD, seguido del número de canal asignado (del 0 al 7).

```
memset(rxBuffer,0,sizeof(rxBuffer));  
HAL_UART_Receive (&huart4, rxBuffer, 512, 1000);  
if(strstr((char *)rxBuffer,"+IPD,0")) channel = 0;  
else if(strstr((char *)rxBuffer,"+IPD,1")) channel = 1;  
else if(strstr((char *)rxBuffer,"+IPD,2")) channel = 2;  
else if(strstr((char *)rxBuffer,"+IPD,3")) channel = 3;  
else if(strstr((char *)rxBuffer,"+IPD,4")) channel = 4;  
else if(strstr((char *)rxBuffer,"+IPD,5")) channel = 5;  
else if(strstr((char *)rxBuffer,"+IPD,6")) channel = 6;  
else if(strstr((char *)rxBuffer,"+IPD,7")) channel = 7;  
else channel = 100;
```

Wi-Fi.

- **Detección de la Petición:** aquí el programa analiza el texto de la petición HTTP para saber qué acción ordenó el navegador:
- Si encuentra GET/lighton: Significa que alguien hizo clic en el botón ON en la página web. Cambia la variable onoff a 0.
- Si encuentra GET /lightoff: Significa que hicieron clic en el botón OFF, cambia onoff a 1.
- Si no encuentra ninguna de las dos: mantiene el estado actual de la luz copiando el valor de la variable led (onoff = led).

Wi-Fi.

- 3. Detección de la Petición: aquí el programa analiza el texto de la petición HTTP para saber qué acción ordenó el navegador:

```
if(strstr((char *)rxBuffer,"GET /lighton")) onoff = 0;  
else if(strstr((char *)rxBuffer,"GET /lightoff")) onoff = 1;  
else onoff = led;
```

Wi-Fi.

- Comandos AT enviados al módulo ESP-01 por UART:
- **AT+CIPSEND=canal,longitud\r\n**: Le indica al ESP-01 que se va a enviar una cantidad específica de bytes (longitud) a través de un canal de conexión TCP activo determinado (canal).

```
if(channel<8 && onoff == 1) {  
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);  
    led = 1;  
    sprintf(ATcommand,"AT+CIPSEND=%d,%d\r\n",channel,countB+countF+countT);  
    memset(rxBuffer,0,sizeof(rxBuffer));  
    HAL_UART_Transmit(&huart4,(uint8_t *)ATcommand,strlen(ATcommand),1000);  
    HAL_UART_Receive (&huart4, rxBuffer, 512, 100);  
}
```

Wi-Fi.

- **AT+CIPCLOSE=canal\r\n:** Qué hace: Cierra de forma ordenada el canal de comunicación TCP una vez que la página web ya fue transmitida al navegador.

```
if(strstr((char *)rxBuffer, ">")) {  
    memset(rxBuffer, 0, sizeof(rxBuffer));  
    HAL_UART_Transmit(&huart4, (uint8_t *)ATcommandB, countB, 1000);  
    HAL_UART_Transmit(&huart4, (uint8_t *)ATcommandF, countF, 1000);  
    HAL_UART_Transmit(&huart4, (uint8_t *)ATcommandT, countT, 1000);  
    HAL_UART_Receive (&huart4, rxBuffer, 512, 100); }  
    sprintf(ATcommand, "AT+CIPCLOSE=%d\r\n", channel);  
    memset(rxBuffer, 0, sizeof(rxBuffer));  
    HAL_UART_Transmit(&huart4, (uint8_t *)ATcommand, strlen(ATcommand), 1000);  
    HAL_UART_Receive (&huart4, rxBuffer, 512, 100); channel=100;  
}
```

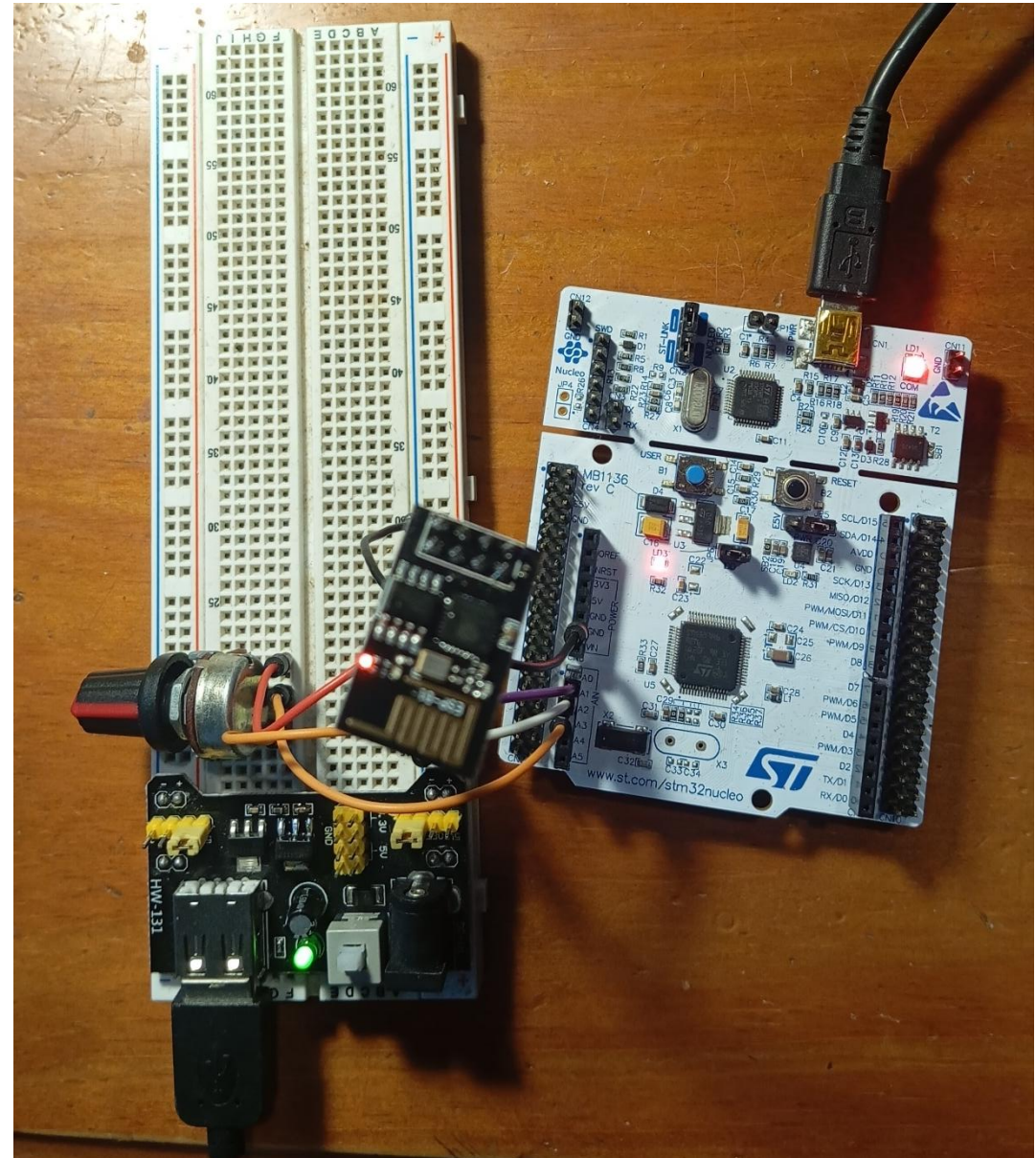
Wi-Fi.

- Experiencia:

STM32 - ESP8266

Light is currently off

ON



Wi-Fi.

- Y si deseo ver los valores del ADC?

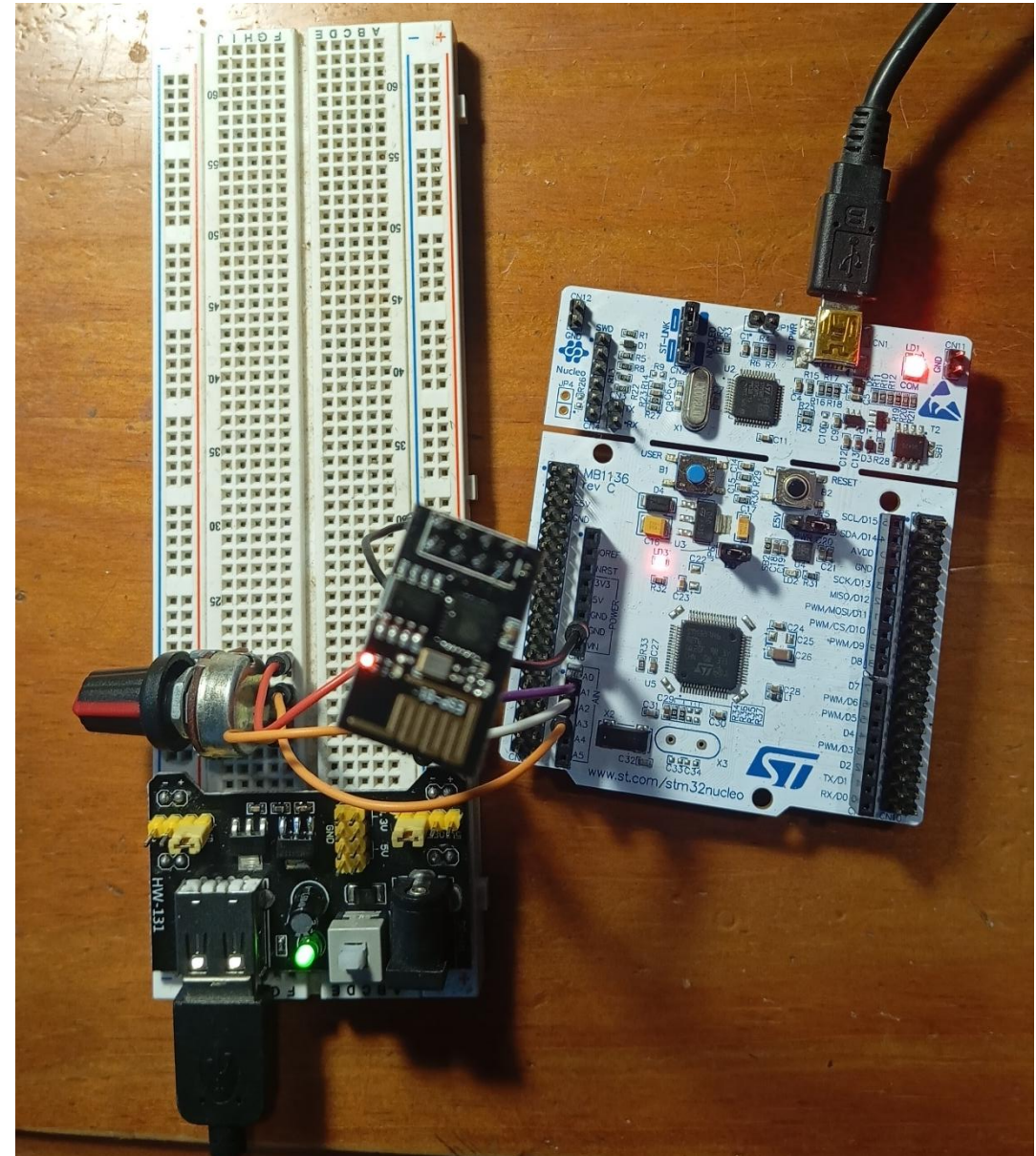
STM32 - ESP8266 WebServer

Tension: V

Valor ADC (0-4095): 1292

Light is currently: **OFF**

ON

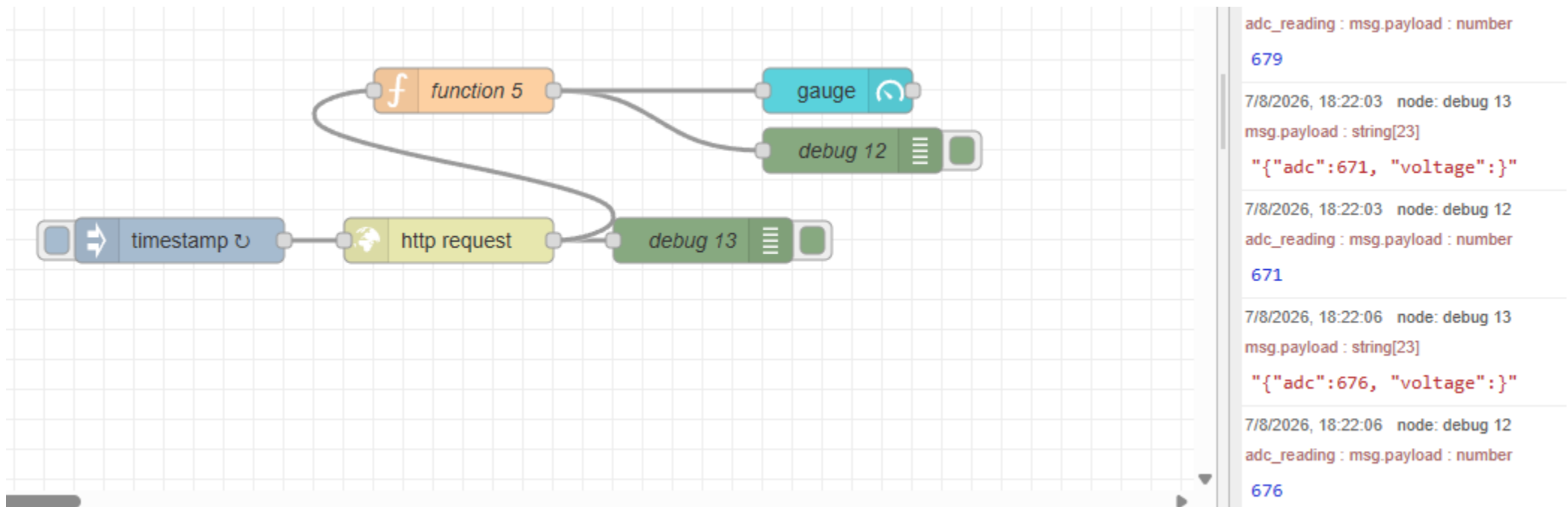


Wi-Fi.

- La etiqueta HTML de auto-refresco:
- HTML
- `<meta http-equiv=\"refresh\" content=\"2\">`
- Le ordena directamente al navegador web que recargue toda la página automáticamente cada 2 segundos.
- Cada vez que pasan esos 2 segundos, el navegador vuelve a hacer una petición al servidor del STM32, lo que obliga al microcontrolador a leer nuevamente el pin analógico.

Wi-Fi.

- Qué necesito si deseo ver los valores de una ADC?



Bibliografía.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media